

Lecture 4: Lifetimes

Practice smart pointers and lifetimes

Willem Vanhulle

DevLab Rust 2025

Tuesday November 25, 2025

github.com/wvhulle/rust-course-ghent

1. Smart pointers (practice)	1
1.1. Recap	2
2. Lifetimes (new)	4
3. Iterators	14

1.1. Recap

1. Smart pointers (practice)

What is a smart pointer?

1.1. Recap

1. Smart pointers (practice)

What is a smart pointer?

A pointer with ownership.

Name 10 smart pointers in Rust.

1.1. Recap

What is a smart pointer?

A pointer with ownership.

Name 10 smart pointers in Rust.

Box<T>, Rc<T>, Arc<T>, RefCell<T>, Mutex<T>, RwLock<T>, Weak<T>, Cow<T>, String, Vec<T>.

A pointer is generally speaking a type that implements Deref trait.

```
1 pub trait Deref {  
2     type Target: ?Sized;  
3     fn deref(&self) -> &Self::Target;  
4 }
```

(playground link)

Which types implement Deref but are NOT smart pointers?

1.1. Recap

What is a smart pointer?

A pointer with ownership.

Name 10 smart pointers in Rust.

Box<T>, Rc<T>, Arc<T>, RefCell<T>, Mutex<T>, RwLock<T>, Weak<T>, Cow<T>, String, Vec<T>.

A pointer is generally speaking a type that implements Deref trait.

```
1 pub trait Deref {  
2     type Target: ?Sized;  
3     fn deref(&self) -> &Self::Target;  
4 }
```

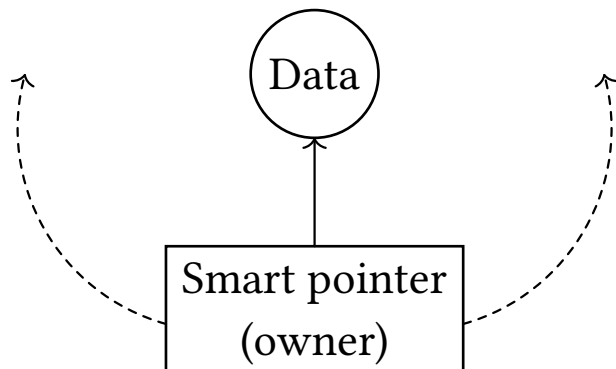
(playground link)

Which types implement Deref but are NOT smart pointers?

References (&T, &mut T) are primitive pointers without ownership. Cell<T> and MaybeUninit<T> don't implement Deref.

1.1. Recap

A smart pointer is a reference / pointer that also owns the data it points to:



Solve the exercises in `session-4/examples/` (if you haven't already) in 30 minutes.

1. Smart pointers (practice)	1
2. Lifetimes (new)	4
2.1. Borrowing with functions	5
2.2. Returning Borrows	6
2.3. Multiple Borrows	7
2.4. Borrow both	8
2.5. Borrow One	9
2.6. Lifetime Elision	10
2.7. Lifetimes in Data Structures	11
2.8. Guidelines	12
2.9. More reading material	13
3. Iterators	14

2.1. Borrowing with functions

2. Lifetimes (new)

```
1  fn borrows(x: &i32) {  
2      dbg!(x);  
3  }  
4  
5  fn main() {  
6      let mut val = 123;  
7  
8      // Borrow `val` for the function call.  
9      borrows(&val);  
10  
11     // Borrow has ended and we're free to mutate.  
12     val += 5;  
13 }
```

(playground link)

```
1  fn identity(x: &i32) -> &i32 {  
2      x  
3  }  
4  
5  fn main() {  
6      let mut x = 123;  
7      let out = identity(&x);  
8      // x = 5; //   `x` is still borrowed!  
9      dbg!(out);  
10 }
```

(playground link)

How does the compiler check out is not a dangling reference?

```
1  fn identity(x: &i32) -> &i32 {  
2      x  
3  }  
4  
5  fn main() {  
6      let mut x = 123;  
7      let out = identity(&x);  
8      // x = 5; //   `x` is still borrowed!  
9      dbg!(out);  
10 }
```

(playground link)

How does the compiler check out is not a dangling reference?

By checking the lifetimes of the references.

Where are the lifetimes?

```
1  fn identity(x: &i32) -> &i32 {  
2      x  
3  }  
4  
5  fn main() {  
6      let mut x = 123;  
7      let out = identity(&x);  
8      // x = 5; //  `x` is still borrowed!  
9      dbg!(out);  
10 }
```

(playground link)

How does the compiler check out is not a dangling reference?

By checking the lifetimes of the references.

Where are the lifetimes?

They are hidden in the signature (**lifetime elision**).

2.3. Multiple Borrows

2. Lifetimes (new)

```
1  fn multiple(a: &i32, b: &i32) -> &i32 {
2      todo!("Return either `a` or `b`")
3  }
4
5  fn main() {
6      let mut a = 5; let mut b = 10;
7      let r = multiple(&a, &b);
8
9      // Which one is still borrowed?
10     // Should either mutation be allowed?
11     a += 7; b += 7;
12     dbg!(r);
13 }
```

(playground link)

Why does this code not compile?

2.3. Multiple Borrows

2. Lifetimes (new)

```
1  fn multiple(a: &i32, b: &i32) -> &i32 {
2      todo!("Return either `a` or `b`")
3  }
4
5  fn main() {
6      let mut a = 5; let mut b = 10;
7      let r = multiple(&a, &b);
8
9      // Which one is still borrowed?
10     // Should either mutation be allowed?
11     a += 7; b += 7;
12     dbg!(r);
13 }
```

(playground link)

Why does this code not compile?

Compiler cannot derive which reference to return, so both are considered borrowed.

2.4. Borrow both

```
1  fn pick<'a>(c: bool, a: &'a i32, b: &'a i32) -> &'a i32 {
2      if c { a } else { b }
3  }
4
5  fn main() {
6      let mut a = 5; let mut b = 10;
7      let r = pick(true, &a, &b);
8      // Which one is still borrowed?
9      // Should either mutation be allowed?
10     // a += 7;
11     // b += 7;
12     dbg!(r);
13 }
```

(playground link)

Which argument is borrowed?

2.4. Borrow both

```
1 fn pick<'a>(c: bool, a: &'a i32, b: &'a i32) -> &'a i32 {
2     if c { a } else { b }
3 }
4
5 fn main() {
6     let mut a = 5; let mut b = 10;
7     let r = pick(true, &a, &b);
8     // Which one is still borrowed?
9     // Should either mutation be allowed?
10    // a += 7;
11    // b += 7;
12    dbg!(r);
13 }
```

(playground link)

Which argument is borrowed?

Both a and b are borrowed for the lifetime of r. Compiler looks only at signature.

2.5. Borrow One

Open example file for demonstration: `find-nearest.rs`

```
1  fn only_args(a: &i32, b: &i32) {  
2      todo!();  
3  }  
4  
5  fn identity(a: &i32) -> &i32 {  
6      a  
7  }  
8  
9  struct Foo(i32);  
10 impl Foo {  
11     fn get(&self, other: &i32) -> &i32  
12     {  
13         &self.0  
14     }  
15 }
```

(playground link)

2.6. Lifetime Elision

```
1  fn only_args(a: &i32, b: &i32) {
2      todo!();
3  }
4
5  fn identity(a: &i32) -> &i32 {
6      a
7  }
8
9  struct Foo(i32);
10 impl Foo {
11     fn get(&self, other: &i32) -> &i32
12     {
13         &self.0
14     }
15 }
```

(playground link)

2.6.1. Rules for lifetime elision

... for references in argument or return position:

- Each argument which does not have a lifetime annotation is given one.
- If there is only **one argument lifetime**, it is given to all un-annotated return values.
- If there are **multiple argument lifetimes**, but the **first one is for self**, that lifetime is given to all un-annotated return values.

Try to complete the missing lifetimes manually in the example code.

2.6. Lifetime Elision

```
1  fn only_args(a: &i32, b: &i32) {
2      todo!();
3  }
4
5  fn identity(a: &i32) -> &i32 {
6      a
7  }
8
9  struct Foo(i32);
10 impl Foo {
11     fn get(&self, other: &i32) -> &i32
12     {
13         &self.0
14     }
15 }
```

(playground link)

2.6.1. Rules for lifetime elision

... for references in argument or return position:

- Each argument which does not have a lifetime annotation is given one.
- If there is only **one argument lifetime**, it is given to all un-annotated return values.
- If there are **multiple argument lifetimes**, but the **first one is for self**, that lifetime is given to all un-annotated return values.

Try to complete the missing lifetimes manually in the example code.

See example file `lifetime-elision.rs`

2.7. Lifetimes in Data Structures

2. Lifetimes (new)

```
1  #[derive(Debug)]
2  enum HighlightColor {
3      Pink,
4      Yellow,
5  }
6
7  #[derive(Debug)]
8  struct Highlight<'document> {
9      slice: &'document str,
10     color: HighlightColor,
11 }
12
13 fn main() {
14     let doc = String::from("The quick brown fox jumps over the lazy dog.");
15     let noun = Highlight { slice: &doc[16..19], color: HighlightColor::Yellow };
16     let verb = Highlight { slice: &doc[20..25], color: HighlightColor::Pink };
17     // drop(doc);
18     dbg!(noun);
19     dbg!(verb);
20 }
```

[\(playground link\)](#)

What does the lifetime 'document represent?

```
1  #[derive(Debug)]
2  enum HighlightColor {
3      Pink,
4      Yellow,
5  }
6
7  #[derive(Debug)]
8  struct Highlight<'document> {
9      slice: &'document str,
10     color: HighlightColor,
11 }
12
13 fn main() {
14     let doc = String::from("The quick brown fox jumps over the lazy dog.");
15     let noun = Highlight { slice: &doc[16..19], color: HighlightColor::Yellow };
16     let verb = Highlight { slice: &doc[20..25], color: HighlightColor::Pink };
17     // drop(doc);
18     dbg!(noun);
19     dbg!(verb);
20 }
```

[\(playground link\)](#)

What does the lifetime 'document represent?

The lifetime of the slice referring to doc String in main() needs to exceed any instance of Highlight (that contains that slice).

2.8. Guidelines

- Types with borrowed data **force users to hold on to the original** data. This can be performant (but harder):
 - creating lightweight views
 - allocation-free parsers
- When possible, make data structures own their data directly.

2.8. Guidelines

- Types with borrowed data **force users to hold on to the original** data. This can be performant (but harder):
 - creating lightweight views
 - allocation-free parsers
- When possible, make data structures own their data directly.

Exercises:

- introductory exercises in Rustlings chapter 16: Lifetimes (clone the Rustlings repo)
- advanced exercise in example file `protobuf-parsing.rs`. (in this session's examples folder)

2.9. More reading material

For those interested: read this Blog post by PretzelHamer titled “Common Rust Lifetime Misconceptions”

Examples of misconceptions (**wrong**):

- T only contains owned types
- if `T: 'static` then T must be valid for the entire program
- `&'a T` and `T: 'a` are the same thing
- my code isn't generic and doesn't have lifetimes
- if it compiles then my lifetime annotations are correct

1. Smart pointers (practice)	1
2. Lifetimes (new)	4
3. Iterators	14
3.1. Motivation	15
3.2. Iterator trait	16
3.3. Questions	17
3.4. Helpers	18

3.1. Motivation

If you want to iterate over the contents of an array, you'll need to define:

- Some state to keep track of where you are in the iteration process, e.g. an index.
- A condition to determine when iteration is done.
- Logic for updating the state of iteration each loop.
- Logic for fetching each element using that iteration state.

```
1 for (int i = 0; i < array_len; i += 1) {  
2     int elem = array[i];  
3 }
```

(playground link)

What is the closest C equivalent of this to Rust?

3.1. Motivation

If you want to iterate over the contents of an array, you'll need to define:

- Some state to keep track of where you are in the iteration process, e.g. an index.
- A condition to determine when iteration is done.
- Logic for updating the state of iteration each loop.
- Logic for fetching each element using that iteration state.

```
1 for (int i = 0; i < array_len; i += 1) {  
2     int elem = array[i];  
3 }
```

(playground link)

What is the closest C equivalent of this to Rust?

It uses a pointer that is incremented instead of an index.

```
1 for (int *ptr = array; ptr < array + len; ptr += 1) {  
2     int elem = *ptr;  
3 }
```

(playground link)

3.2. Iterator trait

The Iterator trait defines how an object can be used to **produce a sequence of values**. For example,

```
1 struct SliceIter<'s> {
2     slice: &'s [i32],
3     i: usize,
4 }
5
6 impl<'s> Iterator for SliceIter<'s> {
7     type Item = &'s i32;
8
9     fn next(&mut self) -> Option<Self::Item> {
10         if self.i == self.slice.len() {
11             None
12         } else {
13             let next = &self.slice[self.i];
14             self.i += 1;
15             Some(next)
16         }
17     }
18 }
19
20 fn main() {
21     let slice = &[2, 4, 6, 8];
22     let iter = SliceIter { slice, i: 0 };
23     for elem in iter {
24         dbg!(elem);
25     }
26 }
```

[\(playground link\)](#)

Why are Rust iterators lazy?

3.3. Questions

Why are Rust iterators lazy?

You can call `next()` to get the next element only when you need it.

Give an example of an infinite iterator.

3.3. Questions

Why are Rust iterators lazy?

You can call `next()` to get the next element only when you need it.

Give an example of an infinite iterator.

`std::iter::repeat(value)` produces an infinite sequence of `value`.

3.4. Helpers

the Iterator trait provides helper methods that can be used to build customized iterators:

```
1 fn main() {  
2     let result: i32 = (1..=10) // Create a range from 1 to 10  
3     .filter(|x| x % 2 == 0) // Keep only even numbers  
4     .map(|x| x * x) // Square each number  
5     .sum(); // Sum up all the squared numbers  
6  
7     println!("The sum of squares of even numbers from 1 to 10 is: {}", result);  
8 }                                     (playground link)
```

What is another name for iterator helpers?

3.4. Helpers

the Iterator trait provides helper methods that can be used to build customized iterators:

```
1 fn main() {  
2     let result: i32 = (1..=10) // Create a range from 1 to 10  
3     .filter(|x| x % 2 == 0) // Keep only even numbers  
4     .map(|x| x * x) // Square each number  
5     .sum(); // Sum up all the squared numbers  
6  
7     println!("The sum of squares of even numbers from 1 to 10 is: {}", result);  
8 } (playground link)
```

What is another name for iterator helpers?

Adapters.

How many adapters are there in the standard library?

3.4. Helpers

the Iterator trait provides helper methods that can be used to build customized iterators:

```
1 fn main() {  
2     let result: i32 = (1..=10) // Create a range from 1 to 10  
3     .filter(|x| x % 2 == 0) // Keep only even numbers  
4     .map(|x| x * x) // Square each number  
5     .sum(); // Sum up all the squared numbers  
6  
7     println!("The sum of squares of even numbers from 1 to 10 is: {}", result);  
8 } (playground link)
```

What is another name for iterator helpers?

Adapters.

How many adapters are there in the standard library?

Approximately 40.